NLP Final Project Proposal

Group 2: Alexander D. Silberman, Anushka Salvi, Chinaza Belolisa, Dania Salman

• What problem did you select and why did you select it?
Clinical notes are long, unstructured, and hard to read and sort by specialty. We plan to build NLP models that classify a medical transcription into its specialty and answer basic questions for instance summarizing a text, assessing complexity of a medical transcription etc. Together, these classification and text generation tasks could speed up triage and documentation.

• What database/dataset will you use?
https://huggingface.co/datasets/DataFog/medical-transcription-instruct

• What NLP methods will you pick from the concept list? Will it be a classical model or will you have to customize it?
- Rule-Based Models
- Recurrent Neural Network
- Autoencoder

Both classical models and customized models will be used and compared.

• What packages are you planning to use? Why?
- pandas, NumPy, matplotlib/seaborn: for data handling and plots.
- spaCy / NLTK: for text preprocessing (tokenization, stopwords, lemmatization) and our rule-based model (keyword and pattern matching for specialties)
- scikit-learn: for classical baselines (TF-IDF with Logistic Regression), train/test splits, and metrics such as accuracy, F1 and confusion matrices.
- PyTorch: to build and train our recurrent network (LSTM/GRU) and autoencoder.
- GUDHI: for persistent homology features

• What NLP tasks will you work on?
Text classification and text generation

• How will you judge the performance of the model? What metrics will you use?
- For text classification: Cohen's Kappa and F1 Score
- For text generation: BiLingual Evaluation Understudy Score (BLEU), Recall Oriented Understudy for Gisting Evaluation (ROUGE), Metric for Evaluation of Translation with Explicit ORdering (METEOR)

• **Provide a rough schedule for completing the project.**
Week 1: Data cleaning and exploratory data analysis + brainstorming ideas for rule-based model, feature engineering, recurrent networks and autoencoder. Test implementation of sample model code from the class repository.

Week 2: Begin the implementation of the established ideas. Improve upon class-based code.

Week 3: Finalize training. Integrate our work together.

Week 4: Polish the repository. Compare to known models. Work on group and individual reports.